

Lab Report No 6
Convolution in Discrete Domain
Digital Signal Processing



Submitted By:

[Huzaifa Shahbaz]

[21-CE-009]

Submitted To:

Engg. Hareem Khan

Dated:

5th March, 2024

Department of Computer Engineering,
HITEC University, Taxila

Lab Example 1:

Solution:

Brief description (3-5 lines)

Convolution Process:

The n dependency of $y[n]$ deserves some care: for each value of n the convolution sum must be computed separately over all values of a dummy variable m each n .

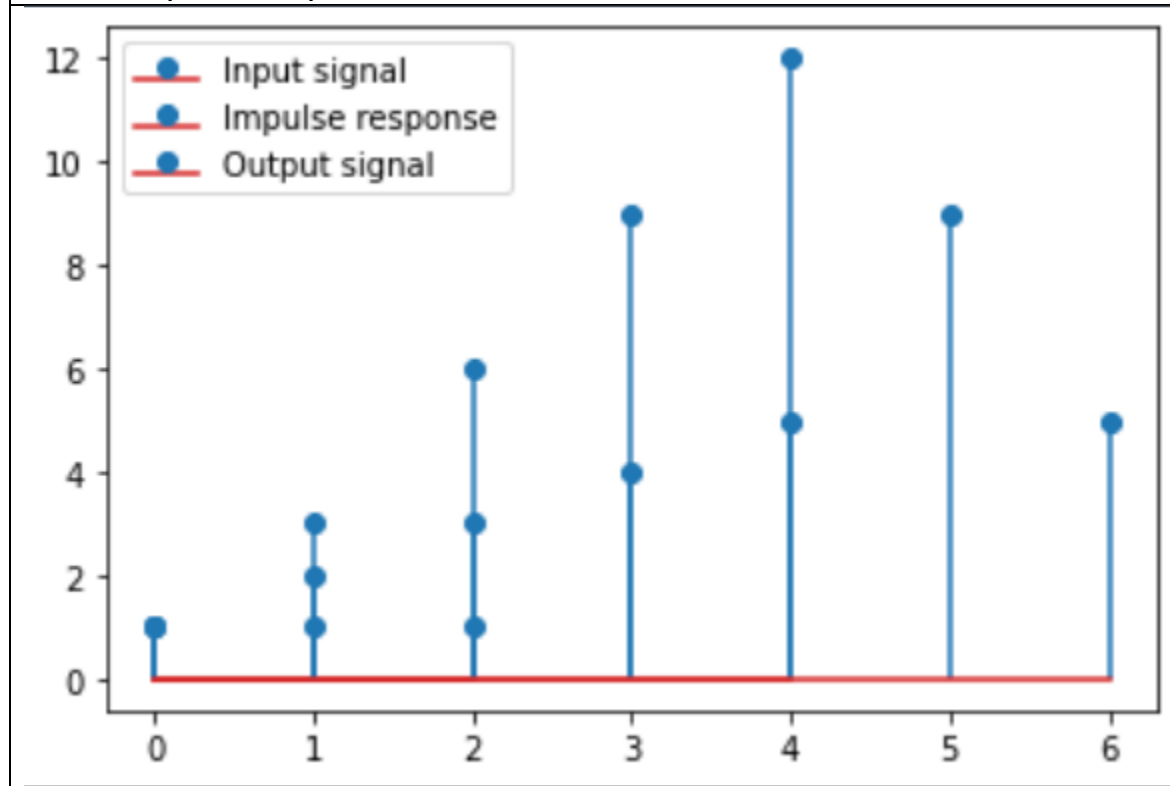
1. Rename the independent variable as m . You now have $x[m]$ and $h[m]$. Flip $h[m]$ over the origin. This is $h[-m]$
2. Shift $h[-m]$ as far left as possible to a point n where the two signals barely touch. This is $h[n-m]$
3. Multiply the two signals and sum over all values of m . This is the convolution sum for the specific n picked above.
4. Shift / move $h[-m]$ to the right by one sample, and obtain a new $h[n-m]$. Multiply and sum over all m . 5. Repeat 2~4 until $h[n-m]$ no longer overlaps with $x[m]$, i.e., shifted out of the $x[m]$ zone.

$$y[n] = x[n] * h[n] = \sum_{m=-\infty}^{\infty} x[m] \cdot h[n-m] = \sum_{m=-\infty}^{\infty} h[m] \cdot x[n-m]$$

The code

```
1  #Convolution
2  import numpy as np
3  import matplotlib.pyplot as plt
4  x = np.array([1, 2, 3, 4, 5])
5  h = np.array([1, 1, 1])
6  def convolve(x, h):
7      y = np.convolve(x, h, mode='full')
8      return y
9  y = convolve(x, h)
10 plt.stem(x, label='Input signal')
11 plt.stem(h, label='Impulse response')
12 plt.stem(y, label='Output signal')
13 plt.legend()
14 plt.show()
```

The results (Screenshot)



Lab Example 2:

Solution:

Brief description (3-5 lines)

Convolution Process:

The n dependency of $y[n]$ deserves some care: for each value of n the convolution sum must be computed separately over all values of a dummy variable m each n .

1. Rename the independent variable as m . You now have $x[m]$ and $h[m]$. Flip $h[m]$ over the origin. This is $h[-m]$
2. Shift $h[-m]$ as far left as possible to a point n where the two signals barely touch. This is $h[n-m]$
3. Multiply the two signals and sum over all values of m . This is the convolution sum for the specific n picked above.
4. Shift / move $h[-m]$ to the right by one sample, and obtain a new $h[n-m]$. Multiply and sum over all m .
5. Repeat 2~4 until $h[n-m]$ no longer overlaps with $x[m]$, i.e., shifted out of the $x[m]$ zone.

$$y[n] = x[n] * h[n] = \sum_{m=-\infty}^{\infty} x[m] \cdot h[n-m] = \sum_{m=-\infty}^{\infty} h[m] \cdot x[n-m]$$

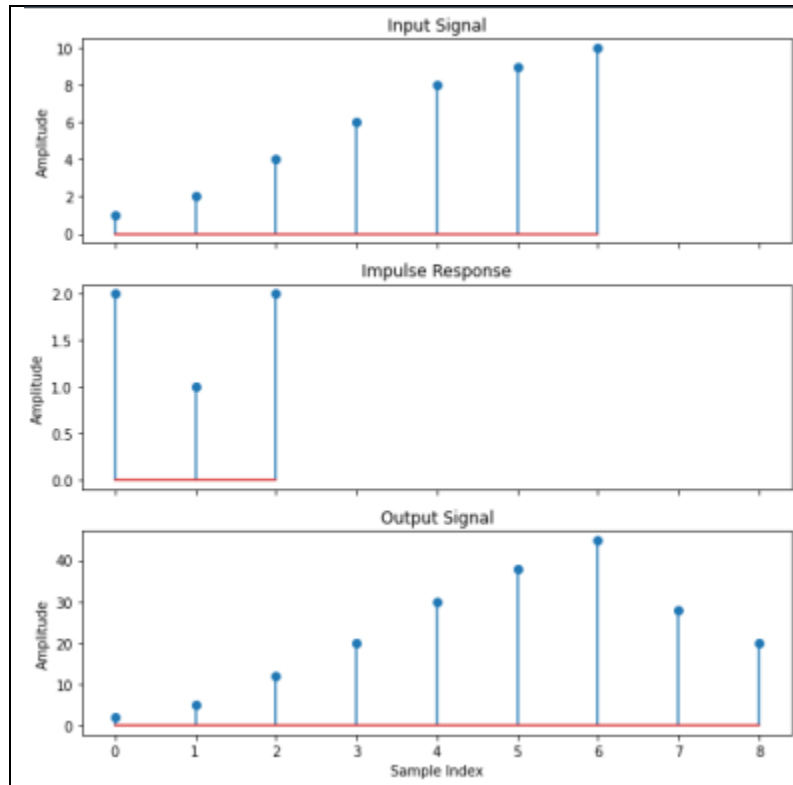
The code

```

1  # Convolution
2  import numpy as np
3  import matplotlib.pyplot as plt
4  x = np.array([1, 2, 4, 6, 8,9,10])
5  h = np.array([2, 1, 2])
6  def convolve(x, h):
7      y = np.convolve(x, h, mode='full')
8      return y
9  y = convolve(x, h)
10 fig, axs = plt.subplots(3, 1, figsize=(8, 8), sharex=True)
11 axs[0].stem(x)
12 axs[0].set_ylabel('Amplitude')
13 axs[0].set_title('Input Signal')
14 axs[1].stem(h)
15 axs[1].set_ylabel('Amplitude')
16 axs[1].set_title('Impulse Response')
17 axs[2].stem(y)
18 axs[2].set_xlabel('Sample Index')
19 axs[2].set_ylabel('Amplitude')
20 axs[2].set_title('Output Signal')
21 plt.tight_layout()
22 plt.show()

```

The results (Screenshot)



Lab Example 3:

Solution:

Brief description (3-5 lines)

Convolution Process:

The n dependency of $y[n]$ deserves some care: for each value of n the convolution sum must be computed separately over all values of a dummy variable m each n .

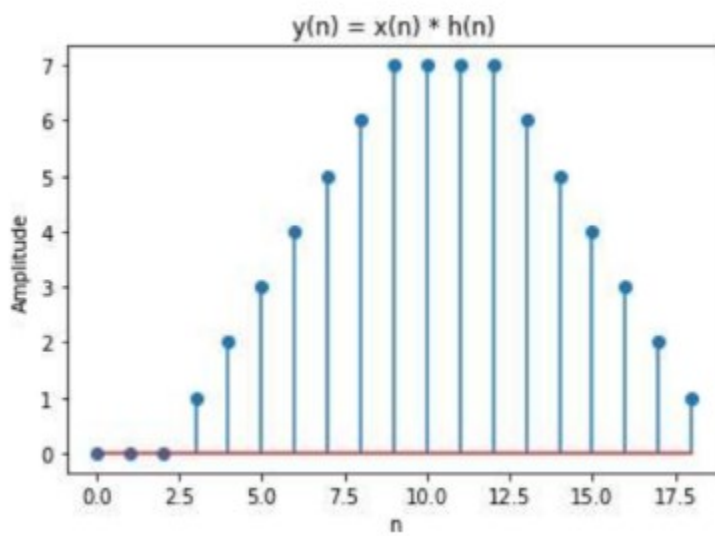
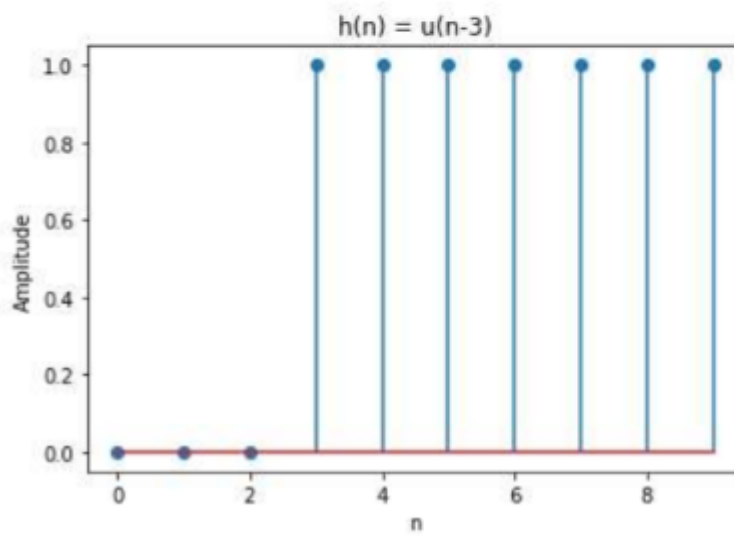
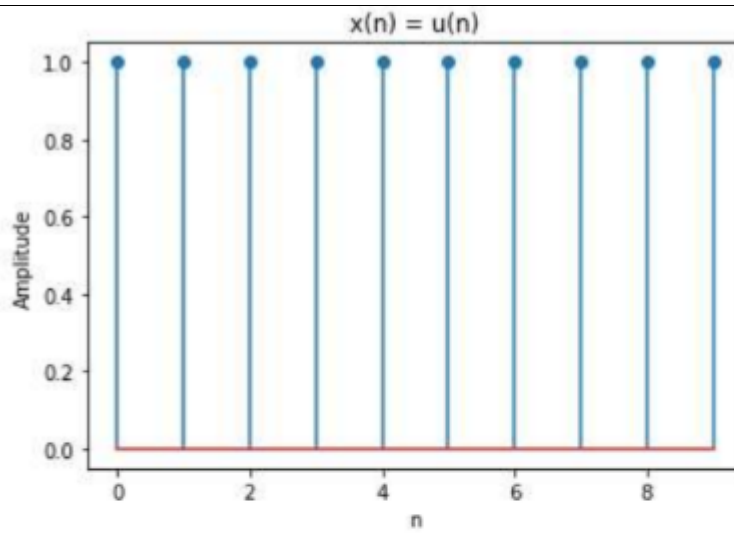
1. Rename the independent variable as m . You now have $x[m]$ and $h[m]$. Flip $h[m]$ over the origin. This is $h[-m]$
2. Shift $h[-m]$ as far left as possible to a point n where the two signals barely touch. This is $h[n-m]$
3. Multiply the two signals and sum over all values of m . This is the convolution sum for the specific n picked above.
4. Shift / move $h[-m]$ to the right by one sample, and obtain a new $h[n-m]$. Multiply and sum over all m .
5. Repeat 2~4 until $h[n-m]$ no longer overlaps with $x[m]$, i.e., shifted out of the $x[m]$ zone.

$$y[n] = x[n] * h[n] = \sum_{m=-\infty}^{\infty} x[m] \cdot h[n-m] = \sum_{m=-\infty}^{\infty} h[m] \cdot x[n-m]$$

The code

```
1  # Convolution
2  import numpy as np
3  import matplotlib.pyplot as plt
4  n = np.arange(0, 10)
5  x = np.heaviside(n, 1)
6  h = np.heaviside(n-3, 1)
7  y = np.convolve(x, h)
8  plt.stem(n, x, use_line_collection=True)
9  plt.title('x(n) = u(n)')
10 plt.xlabel('n')
11 plt.ylabel('Amplitude')
12 plt.show()
13 plt.stem(n, h, use_line_collection=True)
14 plt.title('h(n) = u(n-3)')
15 plt.xlabel('n')
16 plt.ylabel('Amplitude')
17 plt.show()
18 plt.stem(np.arange(0, 19), y, use_line_collection=True)
19 plt.title('y(n) = x(n) * h(n)')
20 plt.xlabel('n')
21 plt.ylabel('Amplitude')
22 plt.show()
```

The results (Screenshot)



Lab Task No 1:

Solution:

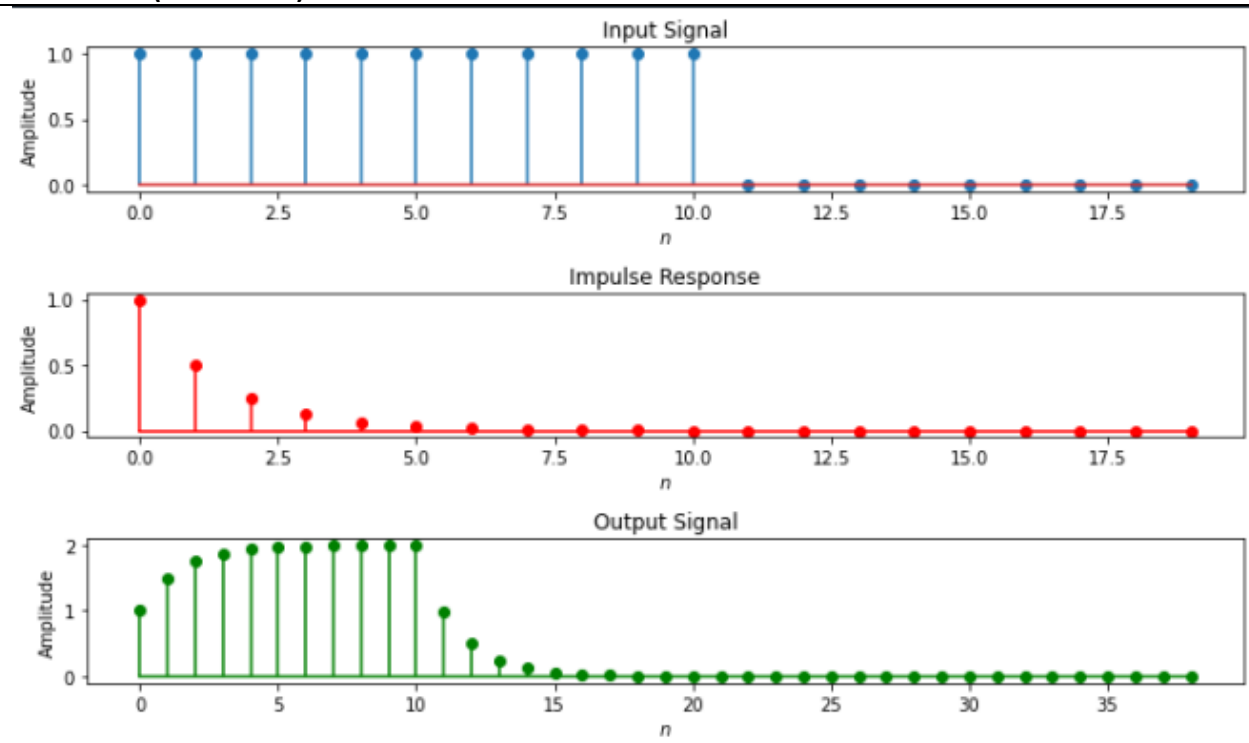
Brief description (3-5 lines)
Consider a system with unit sample response. $h(n) = u(n)a^n$ Find the response to an input $x(n) = u(n) - u(n-N)$ find the convolution.
The code
<pre># Convolution import numpy as np import matplotlib.pyplot as plt def h_n(n, a): return np.where(n >= 0, a ** n, 0) def x_n(n, N): return np.where((n >= 0) & (n <= N), 1, 0) def convolution(x, h): N1 = len(x) N2 = len(h) N = N1 + N2 - 1 y = np.zeros(N) for n in range(N): for k in range(max(0, n - N2 + 1), min(N1, n + 1)): y[n] += x[k] * h[n - k] return y a = 0.5 N = 10 n = np.arange(0, 2 * N) x = x_n(n, N) h = h_n(n, a) y = convolution(x, h) plt.figure(figsize=(10, 6)) plt.subplot(3, 1, 1) plt.stem(n, x, label='\$x(n)\$') plt.xlabel('\$n\$') plt.ylabel('Amplitude') plt.title('Input Signal') plt.subplot(3, 1, 2) plt.stem(n, h, label='\$h(n)\$', linefmt='r-', markerfmt='ro', basefmt='r-')</pre>


```

plt.xlabel('$n$')
plt.ylabel('Amplitude')
plt.title('Impulse Response')
plt.subplot(3, 1, 3)
plt.stem(np.arange(0, len(y)), y, label='$y(n)$', linefmt='g-', markerfmt='go',
basefmt='g-')
plt.xlabel('$n$')
plt.ylabel('Amplitude')
plt.title('Output Signal')
plt.tight_layout()
plt.show()

```

The results (Screenshot)



Lab Task No 2:

Solution:

Brief description (3-5 lines)

Find the convolution of
 $x(n) = (1/2)^n u(n)$,
 $h(n) = u(n) - u(n - 10)$

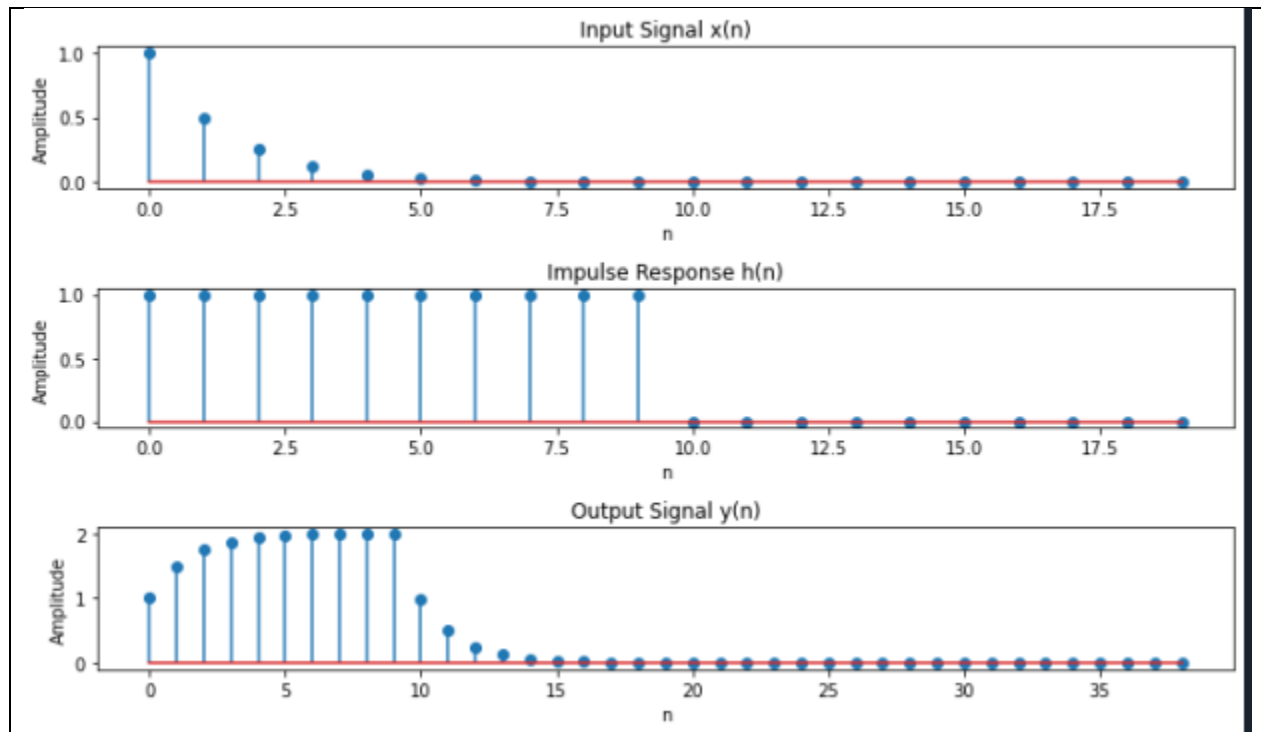
The code

```

# Convolution
import numpy as np
import matplotlib.pyplot as plt
def input_signal(n):
    return (1/2)**n * (n >= 0).astype(int)
def impulse_response(n):
    return ((n >= 0).astype(int)) - ((n - 10) >= 0).astype(int)
def convolution(x, h):
    return np.convolve(x, h, mode='full')
n = np.arange(0, 20)
x_n = input_signal(n)
h_n = impulse_response(n)
y_n = convolution(x_n, h_n)
plt.figure(figsize=(10, 6))
plt.subplot(3, 1, 1)
plt.stem(n, x_n)
plt.title('Input Signal x(n)')
plt.xlabel('n')
plt.ylabel('Amplitude')
plt.subplot(3, 1, 2)
plt.stem(n, h_n)
plt.title('Impulse Response h(n)')
plt.xlabel('n')
plt.ylabel('Amplitude')
plt.subplot(3, 1, 3)
plt.stem(np.arange(0, len(y_n)), y_n)
plt.title('Output Signal y(n)')
plt.xlabel('n')
plt.ylabel('Amplitude')
plt.tight_layout()
plt.show()

```

The results (Screenshot)



Lab Task No 3:

Solution:

Brief description (3-5 lines)

Find the convolution of
 $x(n] = (1/3)^{-n}u(-n-1)$,
 $h(n) = u(n-1)$

The code

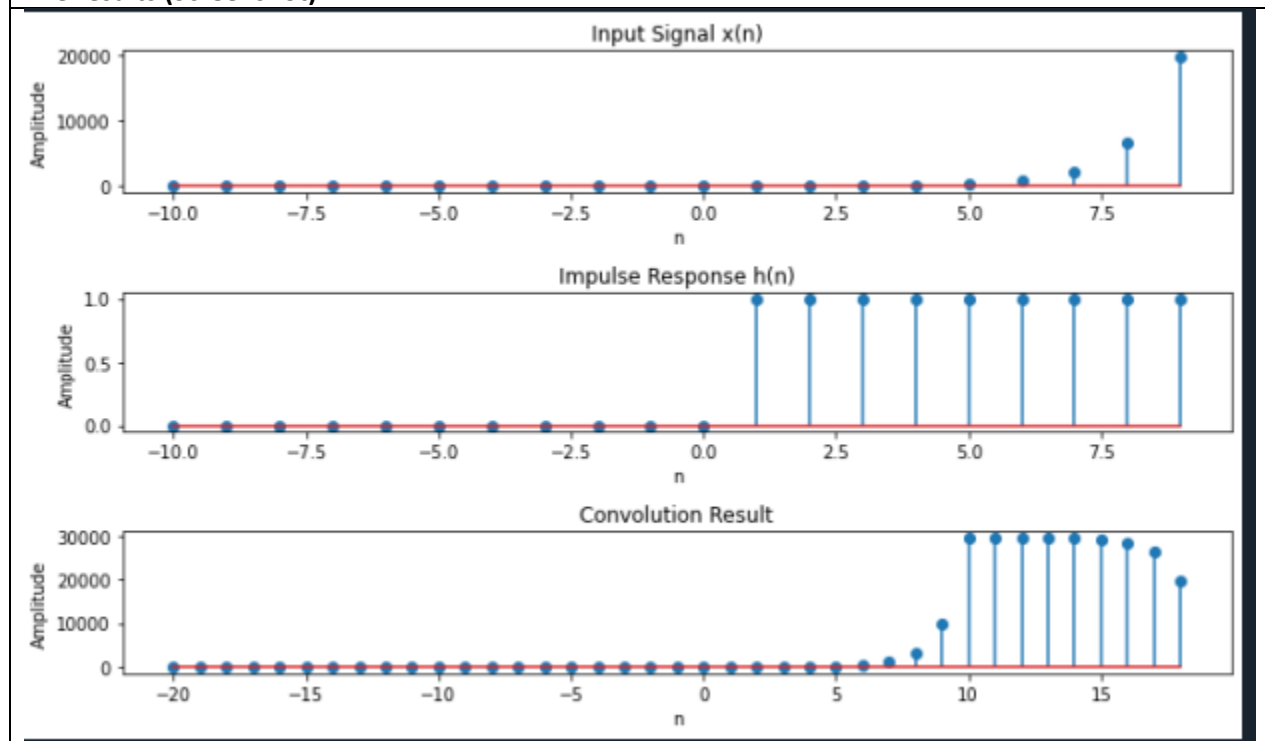
```
# Convolution
import numpy as np
import matplotlib.pyplot as plt
def x(n):
    return (1/3)**(-n) * (n >= 0)
def h(n):
    return (n >= 1)
n = np.arange(-10, 10)
convolution_result = np.convolve(x(n), h(n), mode='full')
plt.figure(figsize=(10, 6))
plt.subplot(3, 1, 1)
plt.stem(n, x(n))
plt.title('Input Signal x(n)')
```

```

plt.xlabel('n')
plt.ylabel('Amplitude')
plt.subplot(3, 1, 2)
plt.stem(n, h(n))
plt.title('Impulse Response h(n)')
plt.xlabel('n')
plt.ylabel('Amplitude')
conv_n = np.arange(n[0]*2, n[-1]*2 + 1)
plt.subplot(3, 1, 3)
plt.stem(conv_n, convolution_result)
plt.title('Convolution Result')
plt.xlabel('n')
plt.ylabel('Amplitude')
plt.tight_layout()
plt.show()

```

The results (Screenshot)



Lab Task No 4:

Solution:

Brief description (3-5 lines)

Find the convolution of

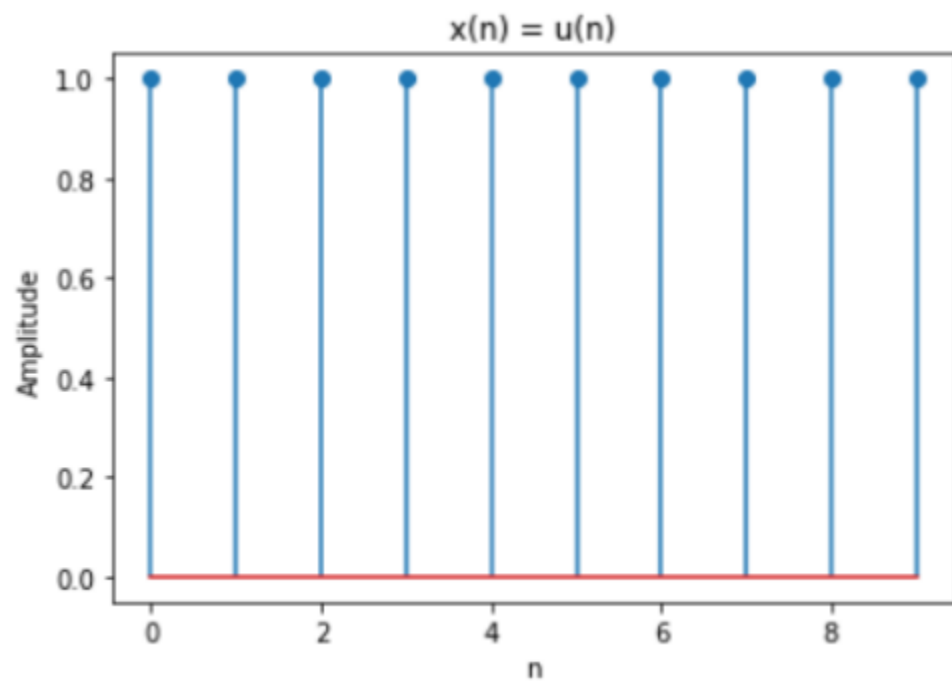
$$x(n) = (1/3)^{-n}u(-n-1),$$

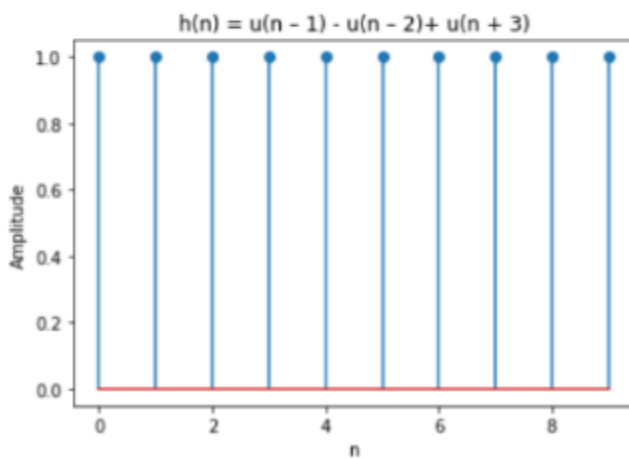
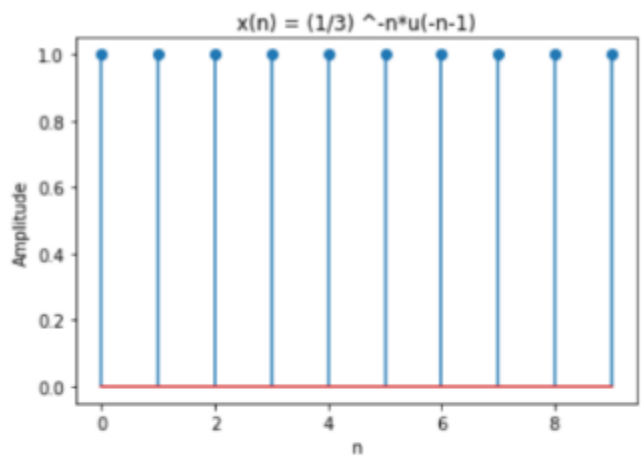
$$h(n) = u(n-1) - u(n-2) + u(n+3)$$

The code

```
1  #lab task 4
2  #program 1
3  import numpy as np
4  import matplotlib.pyplot as plt
5  n = np.arange(0, 10)
6  x = np.heaviside(n, 1)
7  h = np.heaviside(n-3, 1)
8  y = np.convolve(x, h)
9  plt.stem(n, x, use_line_collection=True)
10 plt.title('x(n) = u(n)')
11 plt.xlabel('n')
12 plt.ylabel('Amplitude')
13 plt.show()
14 plt.stem(n, x, use_line_collection=True)
15 plt.title('x(n) = (1/3)^{-n}u(-n-1)')
16 plt.xlabel('n')
17 plt.ylabel('Amplitude')
18 plt.show()
19 plt.stem(n, x, use_line_collection=True)
20 plt.title('h(n) = u(n - 1) - u(n - 2) + u(n + 3)')
21 plt.xlabel('n')
22 plt.ylabel('Amplitude')
23 plt.show()
```

The results (Screenshot)





Lab Task No 5(a):

Solution:

Brief description (3-5 lines)

Find the convolution of
 $y(n) = \sin(n\pi/2) u(n) * u(n-2)$

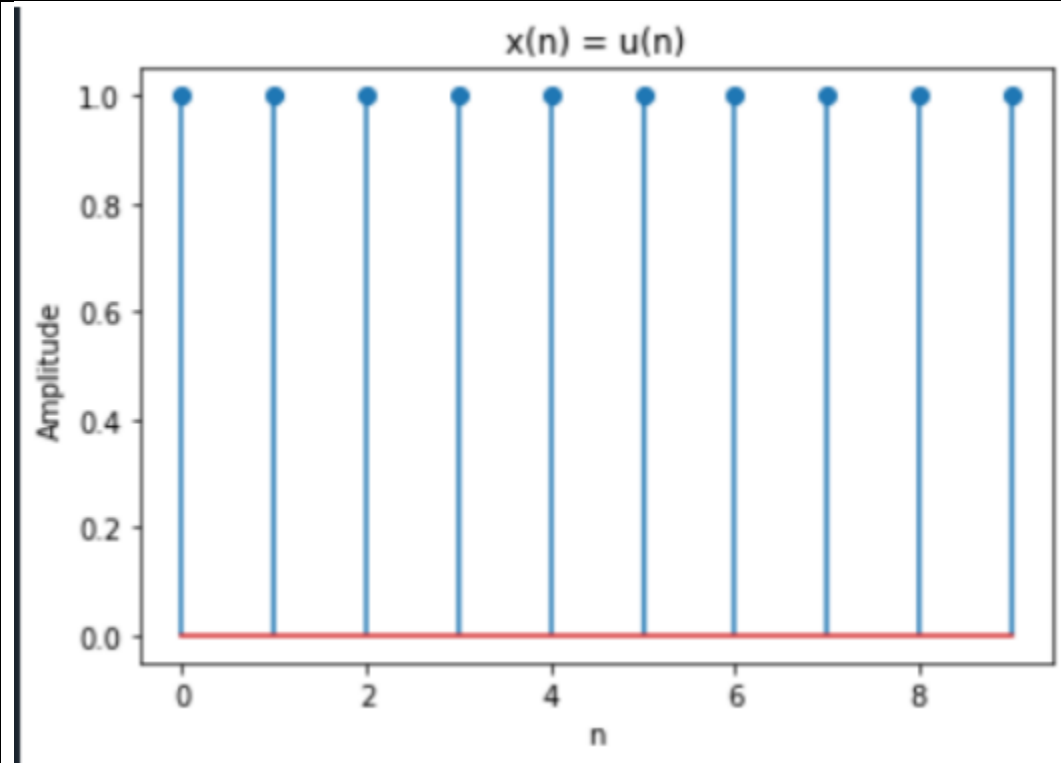
The code

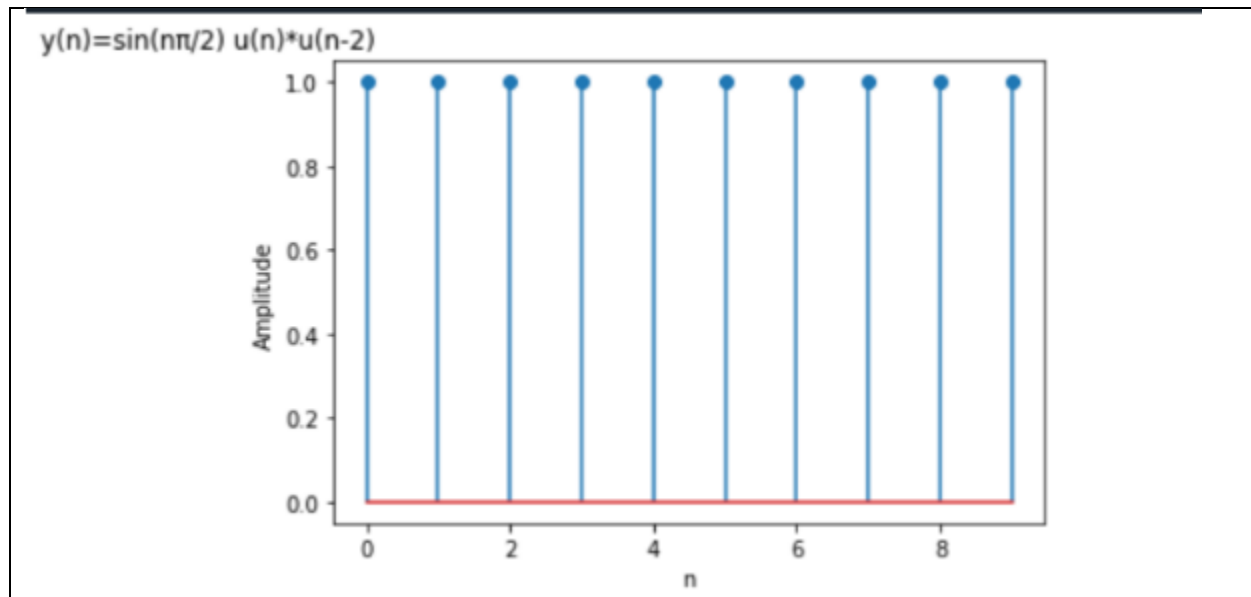
```

1  #lab task 5(a)
2  #program 1
3  import numpy as np
4  import matplotlib.pyplot as plt
5  n = np.arange(0, 10)
6  x = np.heaviside(n, 1)
7  h = np.heaviside(n-3, 1)
8  y = np.convolve(x, h)
9  plt.stem(n, x, use_line_collection=True)
10 plt.title('x(n) = u(n)')
11 plt.xlabel('n')
12 plt.ylabel('Amplitude')
13 plt.show()
14 plt.stem(n, y, use_line_collection=True)
15 plt.title('y(n)=sin(n $\pi$ /2) u(n)*u(n-2)')
16 plt.xlabel('n')
17 plt.ylabel('Amplitude')
18 plt.show()

```

The results (Screenshot)





Lab Task No5(b):

Solution:

Brief description (3-5 lines)

Find the convolution of
 $y(n) = 3^n u(-n+3) * u(n-2)$

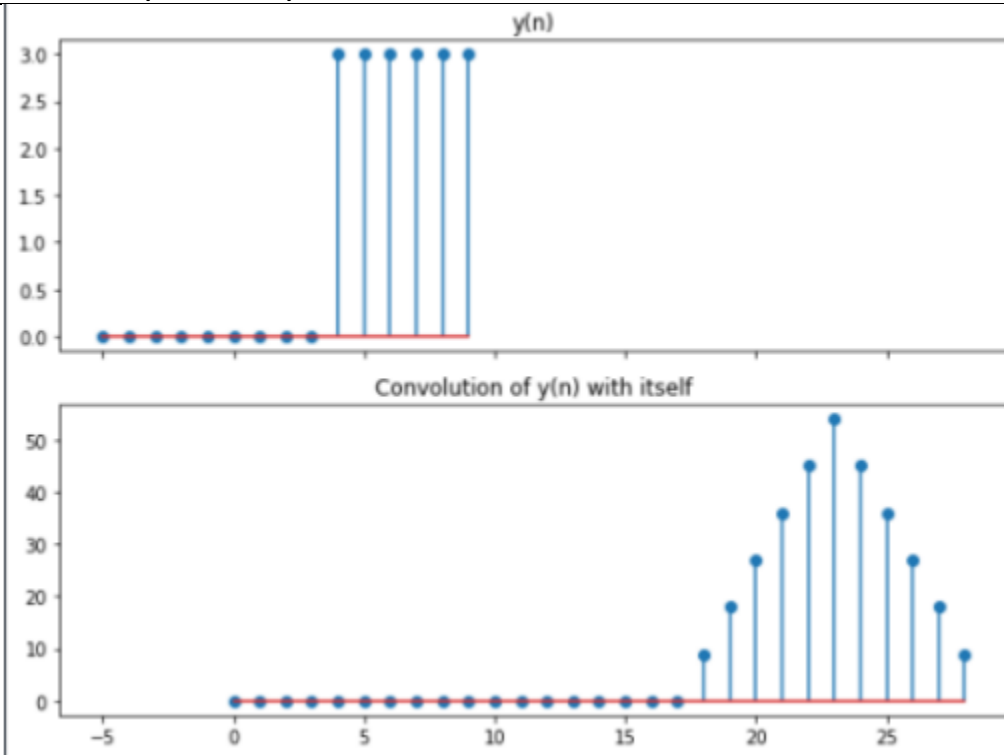
The code

```

1  #lab task 5(b)
2  #program 2
3  import numpy as np
4  import matplotlib.pyplot as plt
5  def u(n):
6      return (n >= 0).astype(int)
7  def nu(n):
8      return (n < 0).astype(int)
9  def y(n):
10     return 3 * nu(-n+3) * u(n-2)
11  n = np.arange(-5, 10)
12  conv = np.convolve(y(n), y(n))
13  fig, (ax1, ax2) = plt.subplots(2, 1, sharex=True, figsize=(8, 6))
14  ax1.stem(n, y(n), use_line_collection=True)
15  ax1.set_title('y(n)')
16  ax2.stem(np.arange(len(conv)), conv, use_line_collection=True)
17  ax2.set_title('Convolution of y(n) with itself')
18  plt.tight_layout()
19  plt.show()

```

The results (Screenshot)



Solution:

Brief description (3-5 lines)

Find the convolution of

$$y(n)=3*nu(-n+3)*u(n-2)$$

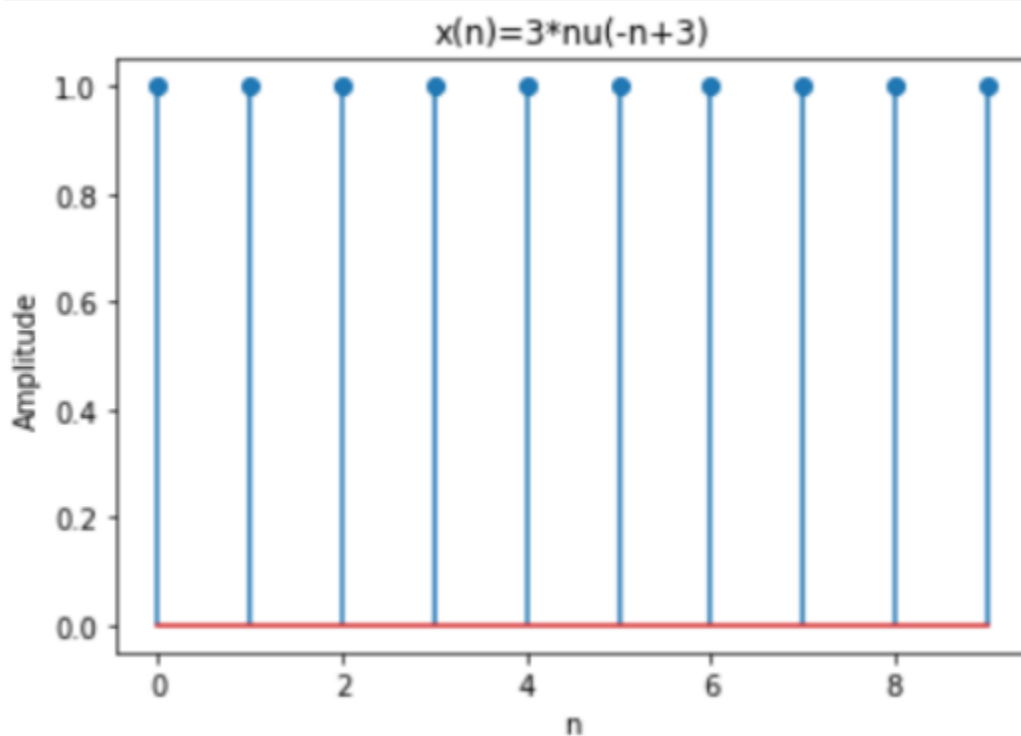
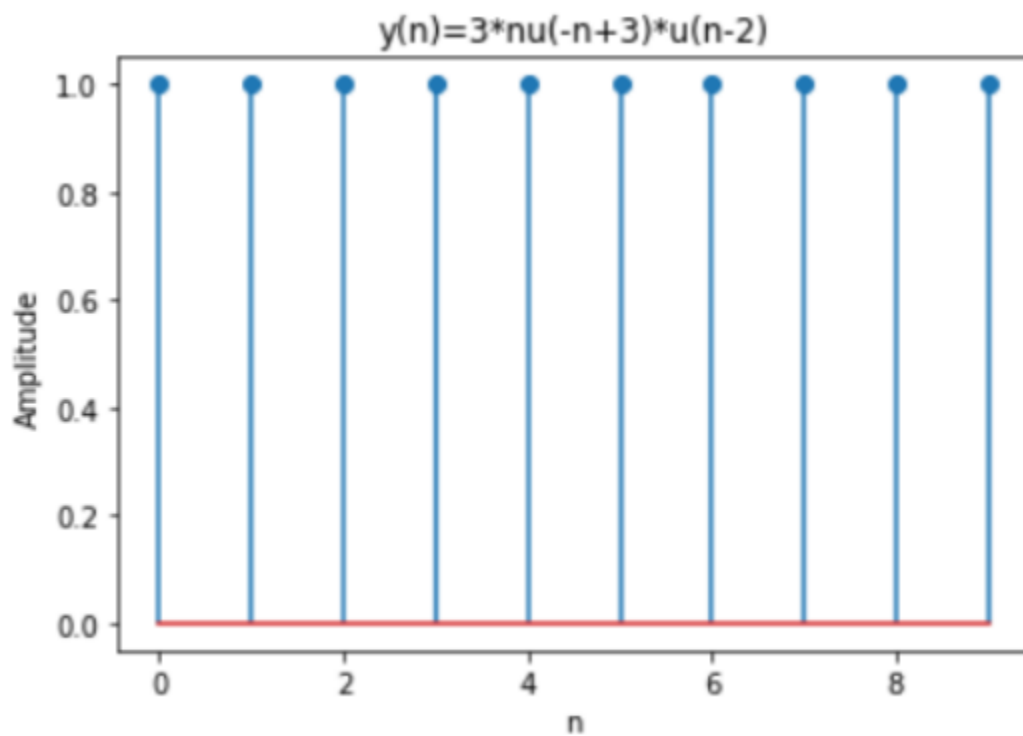
$$x(n)=3*nu(-n+3)$$

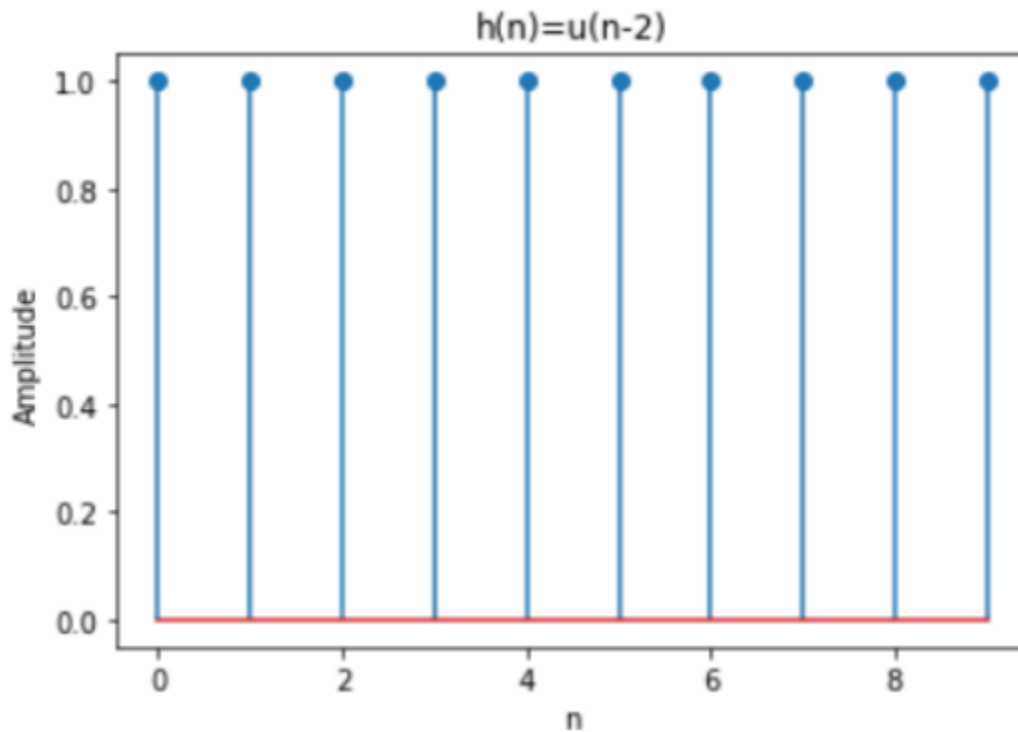
$$h(n)=u(n-2)$$

The code

```
1  #lab task 6
2  #program 1
3  import numpy as np
4  import matplotlib.pyplot as plt
5  n = np.arange(0, 10)
6  x = np.heaviside(n, 1)
7  h = np.heaviside(n-3, 1)
8  y = np.convolve(x, h)
9  plt.stem(n, x, use_line_collection=True)
10 plt.title('y(n)=3*nu(-n+3)*u(n-2)')
11 plt.xlabel('n')
12 plt.ylabel('Amplitude')
13 plt.show()
14 plt.stem(n, x, use_line_collection=True)
15 plt.title('x(n)=3*nu(-n+3)')
16 plt.xlabel('n')
17 plt.ylabel('Amplitude')
18 plt.show()
19 plt.stem(n, x, use_line_collection=True)
20 plt.title('h(n)=u(n-2)')
21 plt.xlabel('n')
22 plt.ylabel('Amplitude')
23 plt.show()
```

The results (Screenshot)



**Conclusion**

In conclusion, convolution in discrete domain is a powerful tool for processing digital data and extracting useful information from it. We also know the working and function of Convolution in Discrete Domain.